

ML Assignment-2

1. GitHub Repository link:

https://github.com/2025ac05971/ML_Assignment_2/

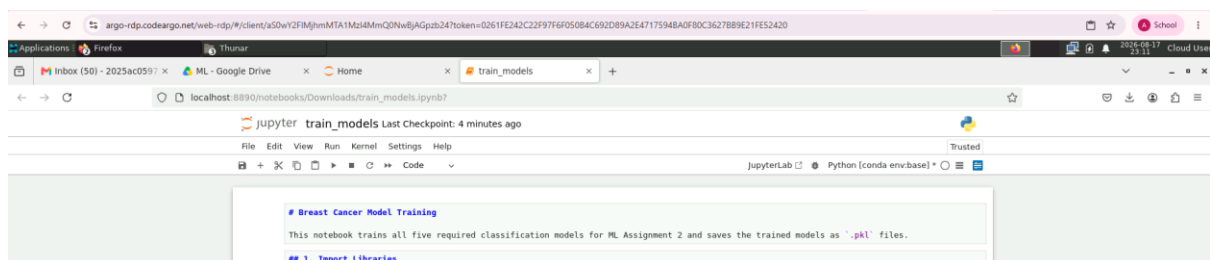
Repository Content:

```
ML_Assignment_2/  
|-- app.py  
|-- requirements.txt  
|-- README.md  
|-- test_data.csv  
|-- data.csv  
|-- model_metrics.csv  
|-- model/  
|   |-- train_models.ipynb  
|   |-- logistic_regression.pkl  
|   |-- decision_tree.pkl  
|   |-- knn.pkl  
|   |-- naive_bayes.pkl  
|   |-- random_forest.pkl  
|   |-- feature_columns.json
```

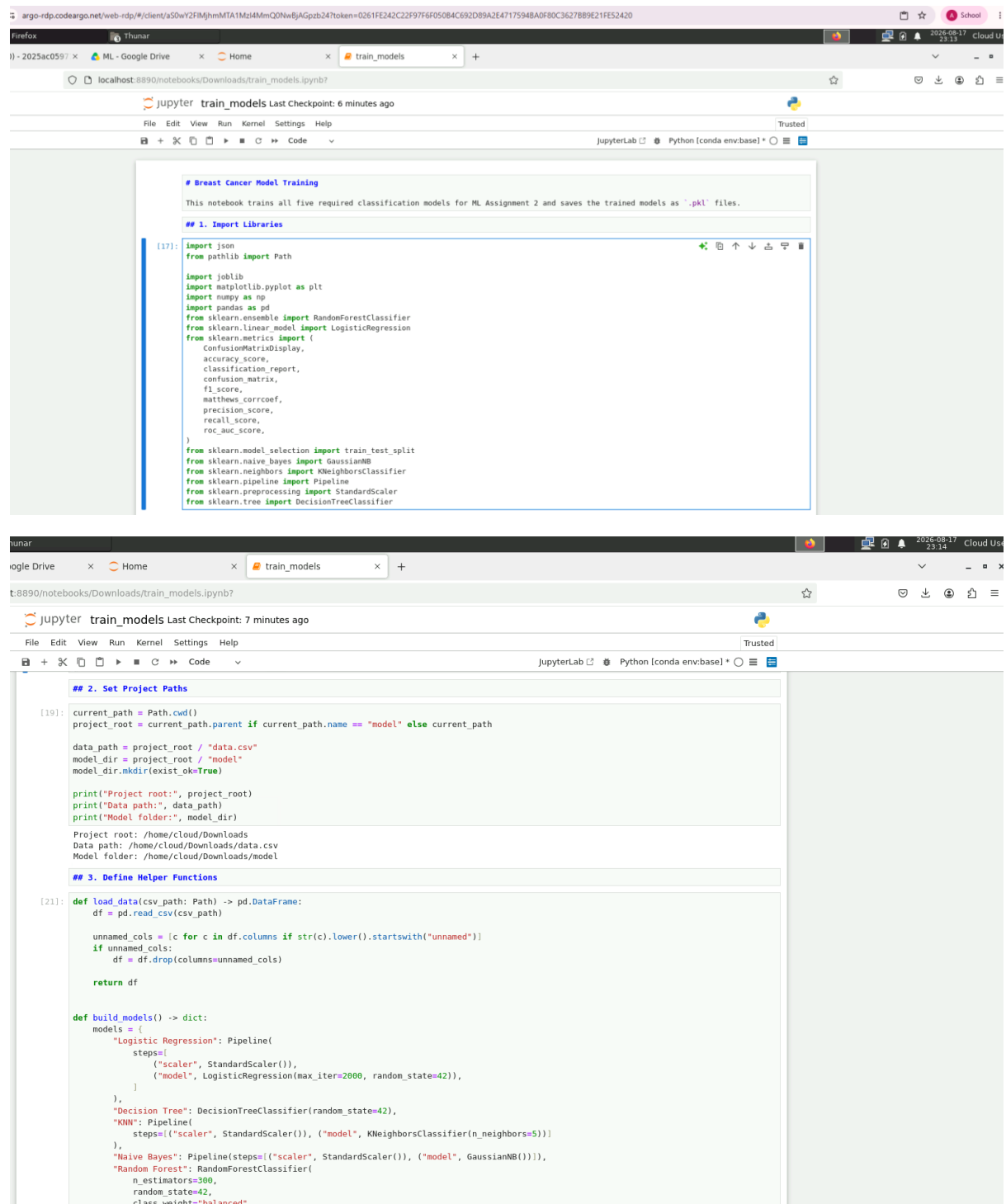
2. Streamlit link:

<https://2025ac05971.streamlit.app/>

3. Virtual lab Snippet:



train_models.ipynb snippet"



```
# Breast Cancer Model Training
This notebook trains all five required classification models for ML Assignment 2 and saves the trained models as '.pkl' files.

## 1. Import Libraries

[17]: import json
from pathlib import Path

import joblib
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    ConfusionMatrixDisplay,
    accuracy_score,
    classification_report,
    confusion_matrix,
    f1_score,
    matthews_corcoef,
    precision_score,
    recall_score,
    roc_auc_score,
)
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier


## 2. Set Project Paths

[19]: current_path = Path.cwd()
project_root = current_path.parent if current_path.name == "model" else current_path

data_path = project_root / "data.csv"
model_dir = project_root / "model"
model_dir.mkdir(exist_ok=True)

print("Project root:", project_root)
print("Data path:", data_path)
print("Model folder:", model_dir)

Project root: /home/cloud/Downloads
Data path: /home/cloud/Downloads/data.csv
Model folder: /home/cloud/Downloads/model


## 3. Define Helper Functions

[21]: def load_data(csv_path: Path) -> pd.DataFrame:
    df = pd.read_csv(csv_path)

    unnamed_cols = [c for c in df.columns if str(c).lower().startswith("unnamed")]
    if unnamed_cols:
        df = df.drop(columns=unnamed_cols)

    return df

def build_models() -> dict:
    models = {
        "Logistic Regression": Pipeline(
            steps=[
                ("scaler", StandardScaler()),
                ("model", LogisticRegression(max_iter=2000, random_state=42)),
            ]
        ),
        "Decision Tree": DecisionTreeClassifier(random_state=42),
        "KNN": Pipeline(
            steps=[("scaler", StandardScaler()), ("model", KNeighborsClassifier(n_neighbors=5))]
        ),
        "Naive Bayes": Pipeline(steps=[("scaler", StandardScaler()), ("model", GaussianNB())]),
        "Random Forest": RandomForestClassifier(
            n_estimators=300,
            random_state=42,
            class_weight="balanced",
```

The image displays three sequential screenshots of a JupyterLab notebook titled "train_models".

Top Screenshot: Shows a code cell with a function `evaluate_model` that takes a model, `x_test`, and `y_test` as input. It returns a dictionary of metrics: Accuracy, AUC, Precision, Recall, F1, and MCC. The function uses `model.predict` and `model.predict_proba` to generate predictions and probabilities.

Middle Screenshot: Shows a code cell titled "## 4. Load Dataset and Prepare Train/Test Split". It loads data from `data_path`, splits it into training and testing sets using `train_test_split`, and prints the total, training, and test record counts. The output shows: Total records: 569, Training records: 455, Test records: 114, and Features used: 38.

Bottom Screenshot: Shows a code cell titled "## 5. Train Models, Evaluate, and Save Outputs". It iterates over different models, trains them, evaluates them using the `evaluate_model` function, and appends the results to a `rows` list. It then saves the metrics to a CSV file, creates a test dataset, and saves it to another CSV file. Finally, it displays the metrics table.

Model	Accuracy	AUC	Precision	Recall	F1	MCC
Logistic Regression	0.964900	0.996000	0.975000	0.928600	0.951200	0.924500
Random Forest	0.964900	0.997000	1.000000	0.904800	0.950000	0.925800
KNN	0.956100	0.982300	0.974400	0.904800	0.938300	0.905800
Decision Tree	0.929800	0.924600	0.904800	0.904800	0.904800	0.849200
Naive Bayes	0.921100	0.989100	0.923100	0.857100	0.888900	0.829200

Jupyter train_models Last checkpoint: 10 minutes ago

File Edit View Run Kernel Settings Help

Python [conda env:base] Trusted

[27]:

```
## 6. Evaluation Metrics, Classification Reports, and Confusion Matrices

for model_name, model in models.items():
    y_pred = model.predict(x_test)
    model_metrics = metrics_df.loc[metrics_df["Model"] == model_name].drop(columns=["Model"])

    print("=" * 70)
    print(f"Evaluation Metrics: {model_name}")
    print("=" * 70)
    display(model_metrics.style.hide(axis="index"))

    print("Confusion Matrix")
    matrix = confusion_matrix(y_test, y_pred, labels=[0, 1])
    display(
        pd.DataFrame(
            matrix,
            index=["Actual Benign", "Actual Malignant"],
            columns=["Predicted Benign", "Predicted Malignant"],
        )
    )

    print("Classification Report")
    print(classification_report(y_test, y_pred, target_names=["Benign", "Malignant"]))

fig, ax = plt.subplots(figsize=(4.5, 3.5))
ConfusionMatrixDisplay.from_predictions(
    y_test,
    y_pred,
    labels=[0, 1],
    display_labels=["Benign", "Malignant"],
    cmap="Greens",
    ax=ax,
)
ax.set_title(f"Confusion Matrix - {model_name}")
plt.tight_layout()
plt.show()
```

Evaluation Metrics: Logistic Regression

Accuracy	AUC	Precision	Recall	F1	MCC
0.964900	0.996000	0.975000	0.928600	0.951200	0.924500

Confusion Matrix

	Predicted Benign	Predicted Malignant
Actual Benign	71	1
Actual Malignant	3	39

Evaluation Metrics: Logistic Regression

Accuracy	AUC	Precision	Recall	F1	MCC
0.964900	0.996000	0.975000	0.928600	0.951200	0.924500

Confusion Matrix

	Predicted Benign	Predicted Malignant
Actual Benign	71	1
Actual Malignant	3	39

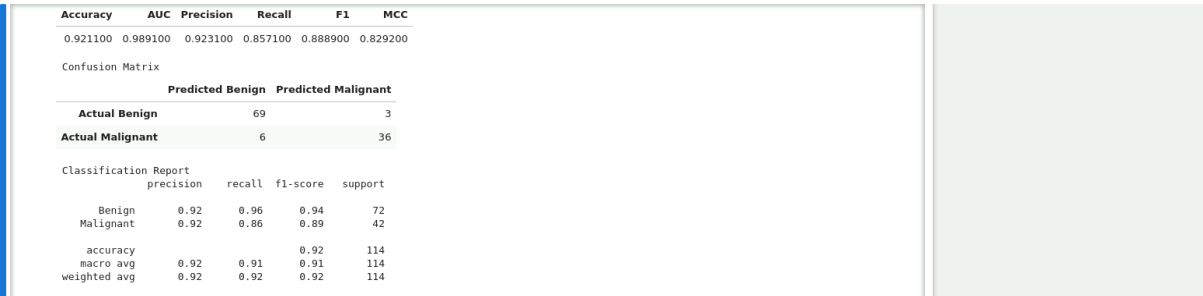
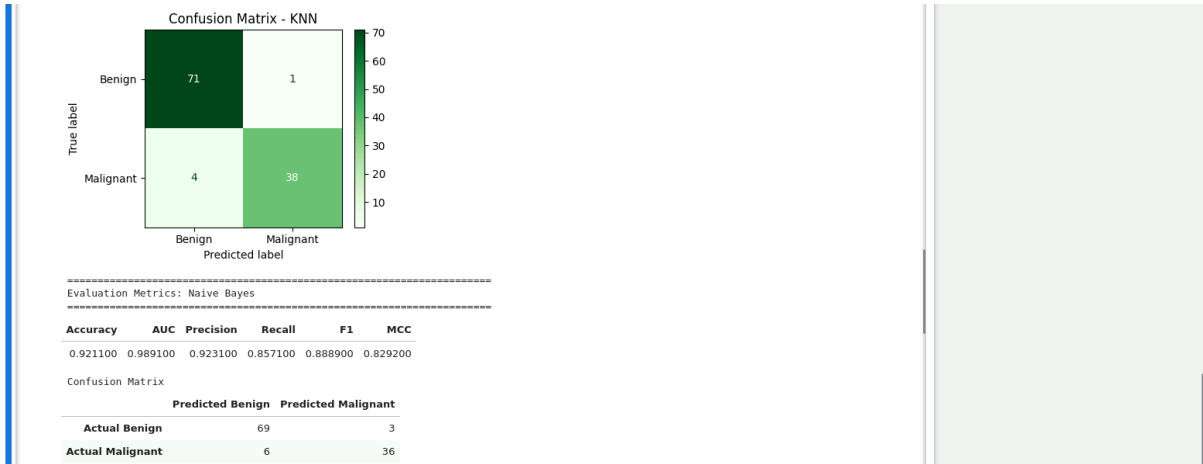
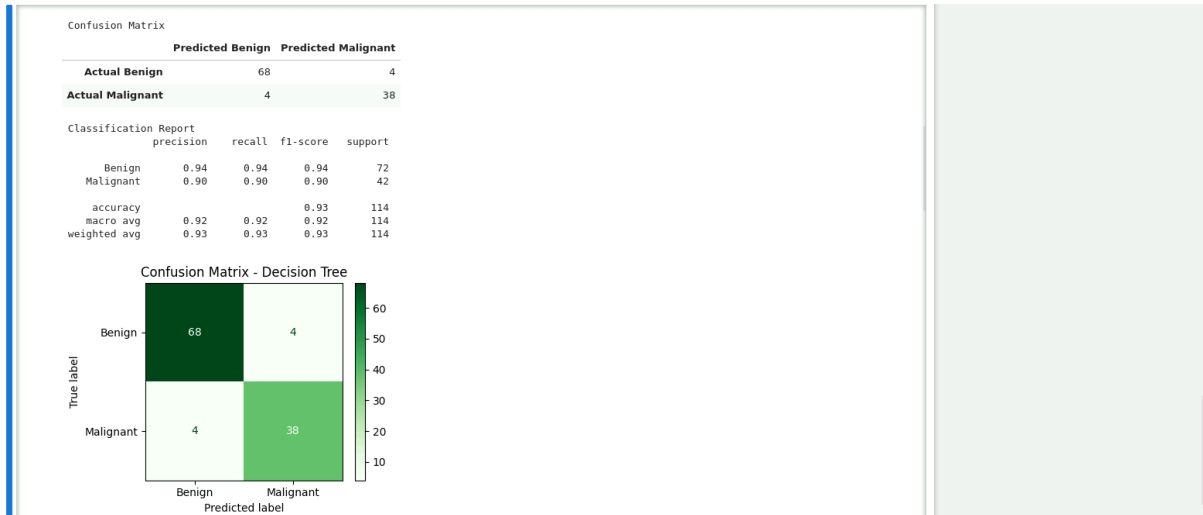
Classification Report

	precision	recall	f1-score	support
Benign	0.96	0.99	0.97	72
Malignant	0.97	0.93	0.95	42
accuracy			0.96	114
macro avg	0.97	0.96	0.96	114
weighted avg	0.97	0.96	0.96	114

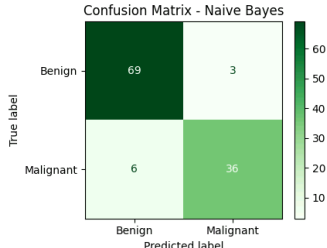
Confusion Matrix - Logistic Regression

Evaluation Metrics: Decision Tree

Accuracy	AUC	Precision	Recall	F1	MCC
0.929800	0.924600	0.904800	0.904800	0.904800	0.849200



Confusion Matrix - Naive Bayes



	Benign	Malignant
Benign	69	3
Malignant	6	36

Evaluation Metrics: Random Forest

Accuracy	AUC	Precision	Recall	F1	MCC
0.964900	0.997000	1.000000	0.904800	0.950000	0.925800

Confusion Matrix

	Predicted Benign	Predicted Malignant
Actual Benign	72	0
Actual Malignant	4	38

Evaluation Metrics: Random Forest

Accuracy	AUC	Precision	Recall	F1	MCC
0.964900	0.997000	1.000000	0.904800	0.950000	0.925800

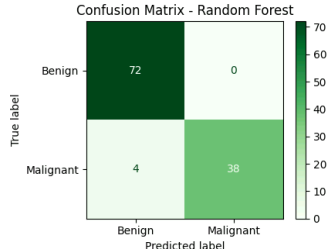
Confusion Matrix

	Predicted Benign	Predicted Malignant
Actual Benign	72	0
Actual Malignant	4	38

Classification Report

	precision	recall	f1-score	support
Benign	0.95	1.00	0.97	72
Malignant	1.00	0.98	0.95	42
accuracy			0.96	114
macro avg	0.97	0.95	0.96	114
weighted avg	0.97	0.96	0.96	114

Confusion Matrix - Random Forest



	Benign	Malignant
Benign	72	0
Malignant	4	38

7. Saved Files

After running this notebook, the following files are generated or updated:

- 'model/logistic_regression.pkl'
- 'model/decision_tree.pkl'
- 'model/knn.pkl'
- 'model/naive_bayes.pkl'
- 'model/random_forest.pkl'
- 'model/feature_columns.json'
- 'model_metrics.csv'
- 'test_data.csv'

4. Streamlit snippet-

2025ac05971.streamlit.app

Install School

Share

Machine Learning Assignment 2

Breast Cancer Wisconsin Classification

This app compares 5 classification models on test data.

1) Upload test data (CSV)

Upload test CSV

test_data.csv

24.2KB

+

File uploaded successfully.

Dataset Information

Training Dataset: 569 instances | Test Dataset: 114 instances (20% split)

Test Data Stats:

Test Instances

114

Features

30

Class Distribution (Test):

- Benign: 72
- Malignant: 42

2) Overall model comparison

Model	Accuracy	AUC	Precision	Recall	F1	MCC
Logistic Regression	0.9649	0.9960	0.9750	0.9286	0.9512	0.9245
Random Forest	0.9649	0.9970	1.0000	0.9048	0.9500	0.9258
KNN	0.9561	0.9823	0.9744	0.9048	0.9383	0.9058
Decision Tree	0.9298	0.9246	0.9048	0.9048	0.9048	0.8492
Naive Bayes	0.9211	0.8891	0.9231	0.8571	0.8889	0.8292

3) Select model

Choose a model

Logistic Regression

Cancel Model Selection

4) Metrics for selected model

Model	Accuracy	AUC	Precision	Recall	F1	MCC
Logistic Regression	0.9649	0.9960	0.9750	0.9286	0.9512	0.9245

5) Confusion matrix

Logistic Regression

True label

Benign (B)

Malignant (M)

Benign (B)

Malignant (M)

Predicted label

71

1

3

39

70

60

50

40

30

20

10

6) Classification report

Label	precision	recall	f1-score	support
Benign	0.9595	0.9861	0.9726	72.0000
Malignant	0.9750	0.9286	0.9512	42.0000
accuracy			0.9649	114.0000
macro avg	0.9672	0.9573	0.9619	114.0000
weighted avg	0.9652	0.9649	0.9647	114.0000

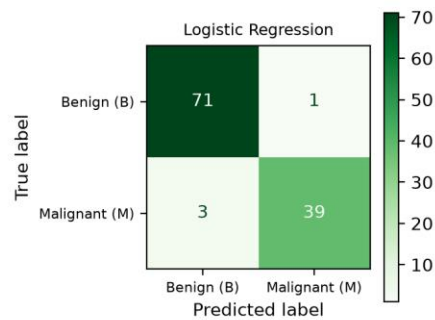
7) Compare selected models with confusion matrix

Choose models to compare

Logistic Regression x Decision Tree x **KNN** x Naive Bayes x Random Forest x

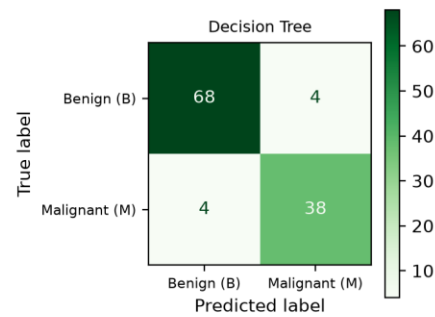
Logistic Regression

Model	Accuracy	AUC	Precision	Recall	F1	MCC	
Logistic Regression		0.9649	0.9960	0.9750	0.9286	0.9512	0.9245



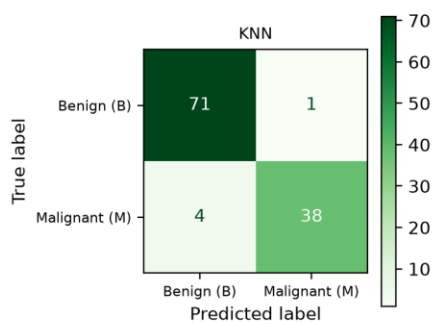
Decision Tree

Model	Accuracy	AUC	Precision	Recall	F1	MCC
Decision Tree	0.9298	0.9246	0.9048	0.9048	0.9048	0.8492



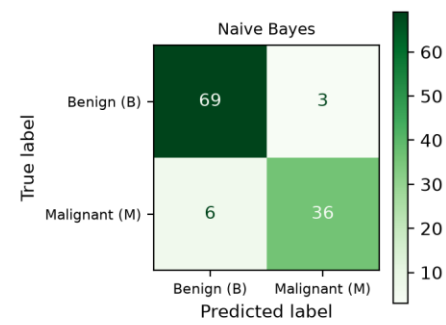
KNN

Model	Accuracy	AUC	Precision	Recall	F1	MCC
KNN	0.9561	0.9823	0.9744	0.9048	0.9383	0.9058



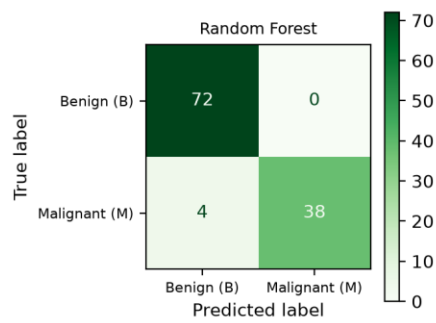
Naive Bayes

Model	Accuracy	AUC	Precision	Recall	F1	MCC
Naive Bayes	0.9211	0.9891	0.9231	0.8571	0.8889	0.8292



Random Forest

Model	Accuracy	AUC	Precision	Recall	F1	MCC
Random Forest	0.9649	0.9970	1.0000	0.9048	0.9500	0.9258



Better model among selected: Random Forest

Ranking rule used: Accuracy, then AUC, then F1, then MCC, then Precision, then Recall.

Model	Accuracy	AUC	Precision	Recall	F1	MCC	
Random Forest		0.9649	0.9970	1.0000	0.9048	0.9500	0.9258
Logistic Regression		0.9649	0.9960	0.9750	0.9286	0.9512	0.9245
KNN		0.9561	0.9823	0.9744	0.9048	0.9383	0.9058
Decision Tree		0.9298	0.9246	0.9048	0.9048	0.9048	0.8492
Naive Bayes		0.9211	0.9891	0.9231	0.8571	0.8889	0.8292

5. README.md snippet-

github.com/2025ac05971/ML_Assignment_2/blob/main/README.md

2025ac05971 / ML_Assignment_2

Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

Files

main

Go to file

model

README.md

app.py

breast-cancer-wisconsin-data-me...

data.csv

model_metrics.csv

requirements.txt

test_data.csv

ML_Assignment_2 / README.md

2025ac05971 Changes done in README.md 106263 · 36 minutes ago History

Preview Code Blame 94 lines (81 loc) · 3.29 KB

Machine Learning Assignment 2

Name: Ankit Nainwal
BITS ID: 2025AC05971

a) Problem statement

This project builds a binary classification system to classify breast cancer tumors as Malignant (M) or Benign (B). Five machine learning models are trained and evaluated on the same dataset using the required metrics.

b) Dataset description

- Dataset name: Breast Cancer Wisconsin (Diagnostic) Data Set
- Source: Kaggle - <https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data>
- Total records: 569
- Features used: 30 numeric features
- Target column: diagnosis (M = malignant, B = benign)
- Problem type: Binary Classification

Files used:

- data.csv for model training and evaluation
- test_data.csv for Streamlit upload/testing

Preview Code Blame 94 lines (81 loc) · 3.29 KB

c) Github Repository Link

- Repository: https://github.com/2025ac05971/ML_Assignment_2

Mandatory files in repo:

- app.py
- requirements.txt
- README.md
- test_data.csv
- model/ (trained model files and training notebook)

Preview Code Blame 94 lines (81 loc) · 3.29 KB

d) Models used

Implemented models:

- Logistic Regression
- Decision Tree Classifier
- K-Nearest Neighbors (KNN)
- Naive Bayes (GaussianNB)
- Random Forest (Ensemble)

Preview Code Blame 94 lines (81 loc) · 3.29 KB

Comparison Table (Required Metrics)

Model	Accuracy	AUC	Precision	Recall	F1	MCC
Random Forest	0.9649	0.9970	1.0000	0.9048	0.9500	0.9258
Logistic Regression	0.9649	0.9960	0.9750	0.9286	0.9512	0.9245
KNN	0.9561	0.9823	0.9744	0.9048	0.9383	0.9058
Decision Tree	0.9298	0.9246	0.9048	0.9048	0.9048	0.8492
Naive Bayes	0.9211	0.9891	0.9231	0.8571	0.8889	0.8292

Preview Code Blame 94 lines (81 loc) · 3.29 KB

Observations on Model Performance

ML Model Name	Observation about model performance
Logistic Regression	Very strong performance with balanced precision and recall. Good general baseline model.
Decision Tree	Easy to interpret, but lower performance than other models on this dataset.
KNN	Performs well after feature scaling, but slightly lower than Logistic Regression and Random Forest.
Naive Bayes	Good AUC score, but lower recall and MCC compared to other models.
Random Forest (Ensemble)	Best overall balance with highest AUC, best precision, and top MCC score.
Best performing model	Random Forest (Ensemble)

Preview Code Blame 94 lines (81 loc) · 3.29 KB

Project structure

```
ML_Assignment_2/  
|-- app.py  
|-- requirements.txt  
|-- README.md  
|-- test_data.csv  
|-- data.csv  
|-- model_metrics.csv  
|-- model/  
|   |-- train_models.ipynb  
|   |-- logistic_regression.pkl  
|   |-- decision_tree.pkl  
|   |-- knn.pkl  
|   |-- naive_bayes.pkl  
|   |-- random_forest.pkl  
|   |-- feature_columns.json
```

Streamlit Deployment Link

- <https://2025ac05971.streamlit.app/>

Streamlit App Features

- CSV file upload with validation
- Model comparison table (all 5 models, 6 metrics)
- Model selection with default model and cancel button
- Individual model metrics display
- Confusion matrix visualization
- Classification report with labels
- Multi-model comparison functionality
- Model ranking by performance
- Dataset information display
 - Training data: 569 instances
 - Test data: 114 instances (20% split)
 - Features count: 30
 - Class distribution: Benign/Malignant